Software Architecture and Process Models
INTE 22293- Academic Year 2024/2025
Session 3: System Thinking & Design Thinking

University of Kelaniya, Sri Lanka

Dr. Dilani Wickramaarachchi

PhD, MSc., BSc. (Special)

Senior Lecturer

Department of Industrial Management, Faculty of Science

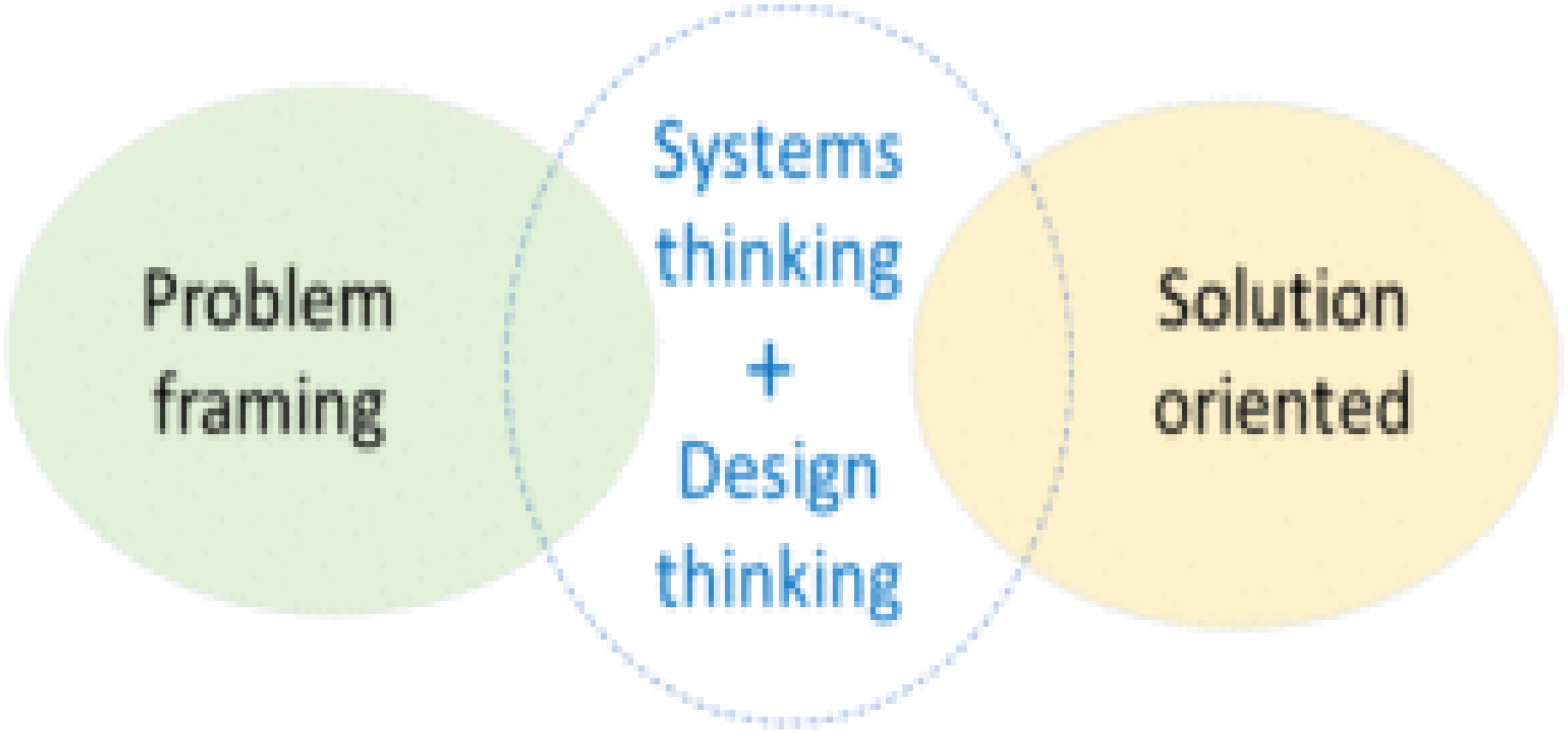
E-Mail – dilani@kln.ac.lk

Session Overview

- Traditional thinking and **system thinking**
- Moving from problem space to solution space
- Design thinking beyond digital
- Design thinking mindsets
- Right team to develop new ideas
- Divergent thinking and convergent thinking



Problem vs Solution

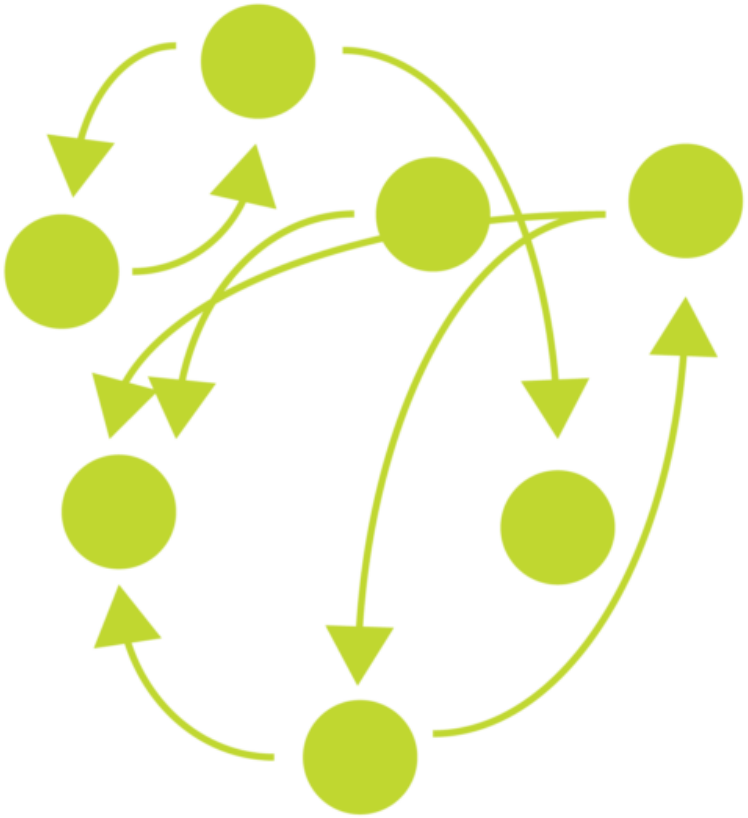


- SDLC

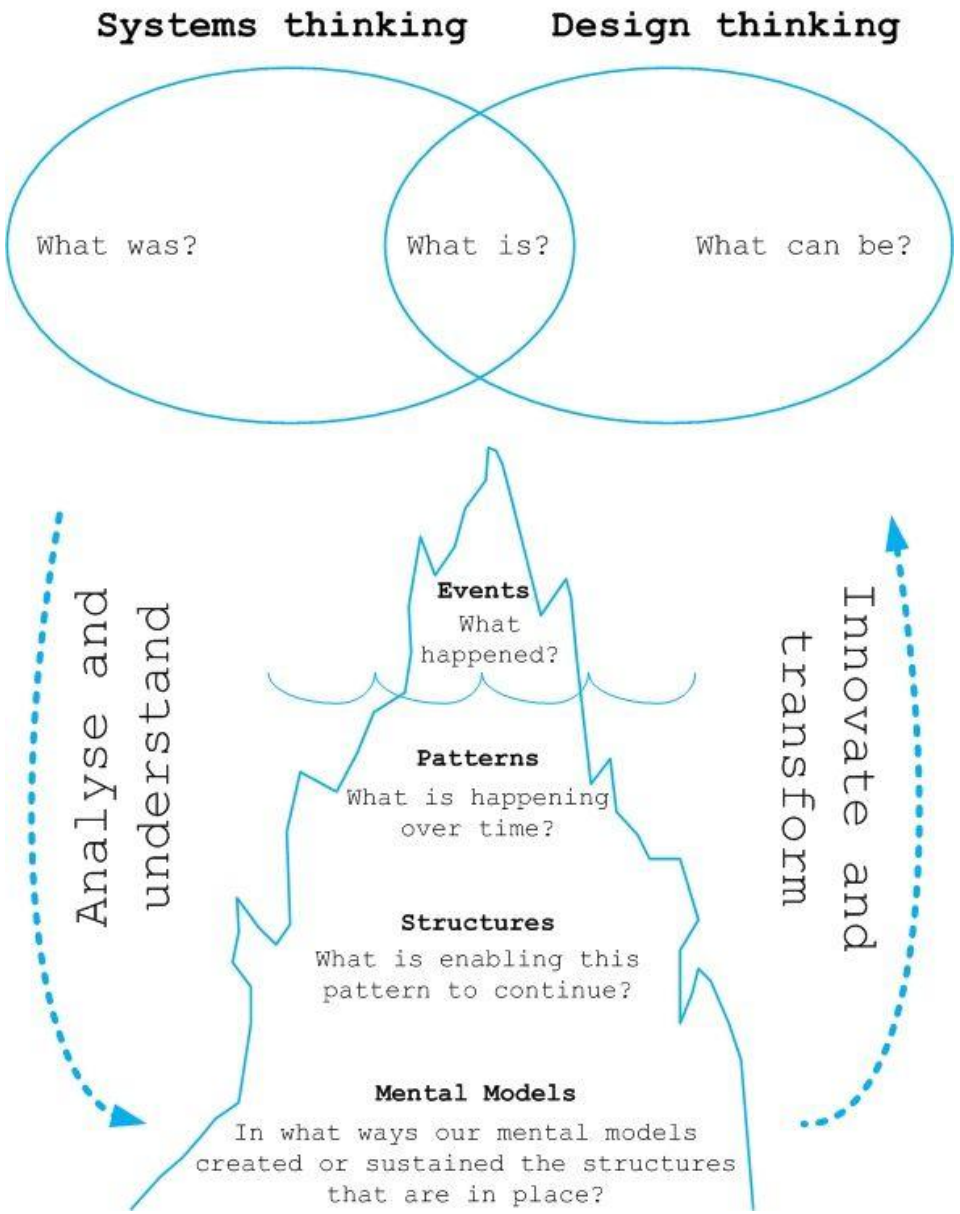
Traditional thinking



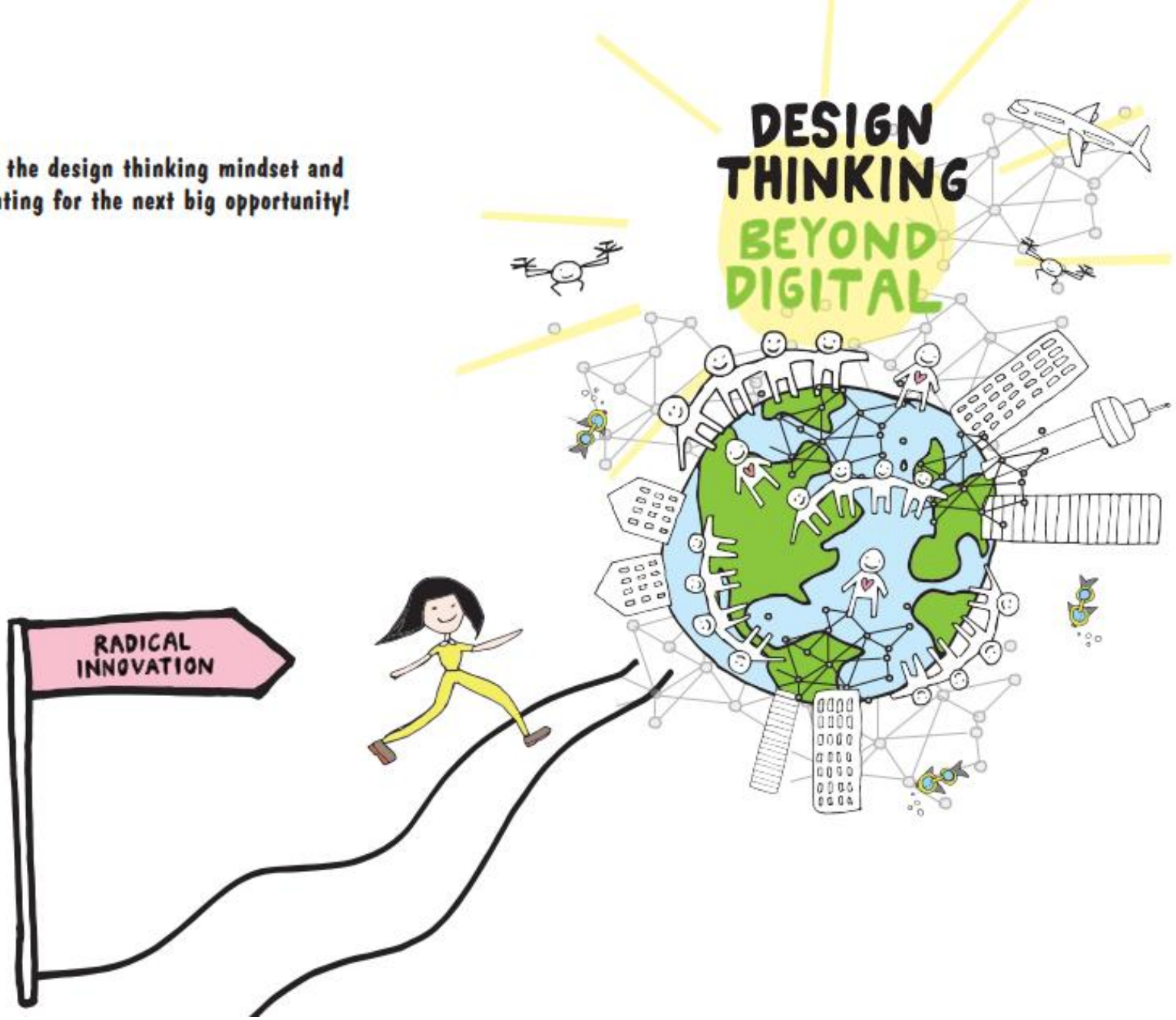
Systems thinking



- Systems thinking asks us to look **beyond the surface-level** "fixes" toward the ***network of underlying causes and effects***.


Systems thinking
Design thinking
What was?
What is?
What can be?
Analyse and understand
Events
What happened?
Patterns
What is happening over time?
Structures
What is enabling this pattern to continue?
Mental Models
In what ways our mental models created or sustained the structures that are in place?
Innovate and transform

Keep up the design thinking mindset and start hunting for the next big opportunity!



Driven by curiosity



We are curious, open, ask "WH questions" continuously, and change the perspective in order to look at things from various sides.

Focus on the people

We focus on the human being, build empathy, and are mindful when exploring his/her needs.



Accept complexity

We explore the key to complex systems, and accept uncertainty and the fact that complex system problems demand complex solutions.



THE DESIGN THINKING PLAYBOOK

MINDSET



Visualize and show

We use stories, visualizations, and simple language to share our findings with the team or create a clear value proposition for our users.



Experiment and iterate

We build and test prototypes iteratively to understand, learn, and solve problems in the context of the user.



Co-create, grow, and scale



We continuously expand our capabilities to create scalable market opportunities in a digital world and in ecosystems.

With varying mental states

As the situation requires, we combine different approaches with design thinking, data analytics, systems thinking, and lean start-up.



NEW MINDSET.
NEW PARADIGM.
BETTER SOLUTIONS.

WWW.DESIGN-THINKING-PLAYBOOK.COM

Develop process awareness

We know where we stand in the design thinking process and develop a feeling for the "groan zone" to change the mindset through facilitation in a targeted way.



Networked collaboration

We collaborate on an ad hoc, agile, and networked basis with T-shaped people and U-shaped teams across departments and companies.

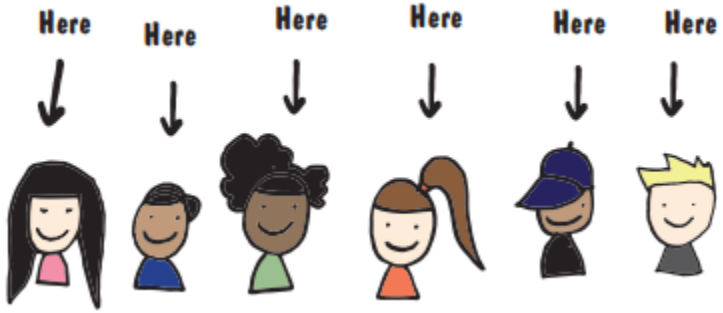


Reflect on actions

We reflect on our way of thinking, our actions, and attitudes because they have an impact on what we do and on the assumptions we make.



You have the right team/right skills here to develop new ideas…



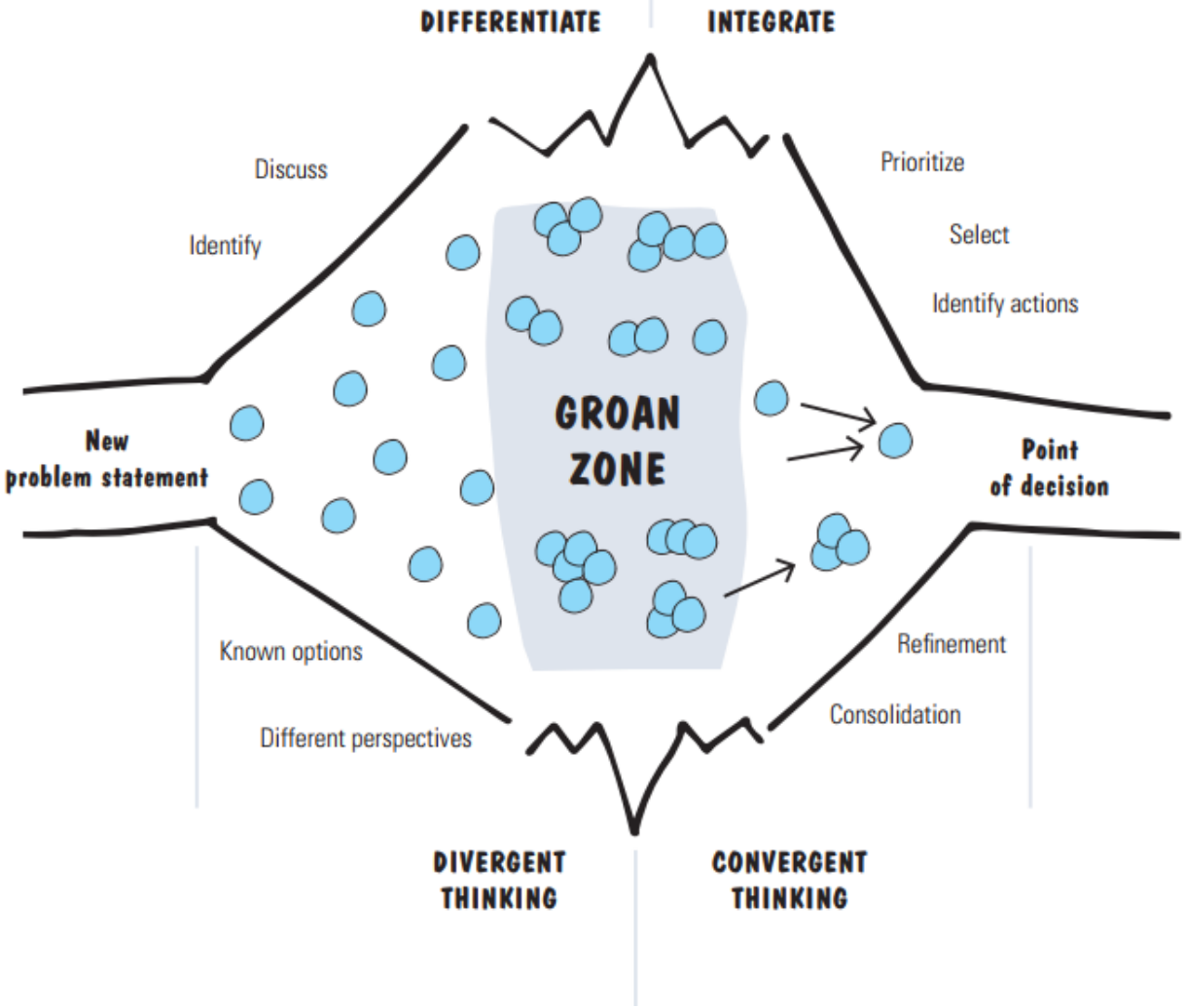
- It's people like us who develop new ideas
  - with great energy and
  - tireless commitment,
  - our heads filled with visions.
- To succeed, we usually
  - need a (customer) need;
  - an interdisciplinary team;
  - the right mindset;
  - and the necessary leeway for experimentation,
  - creativity,
  - courage to question what exists

Three things.. important to us..

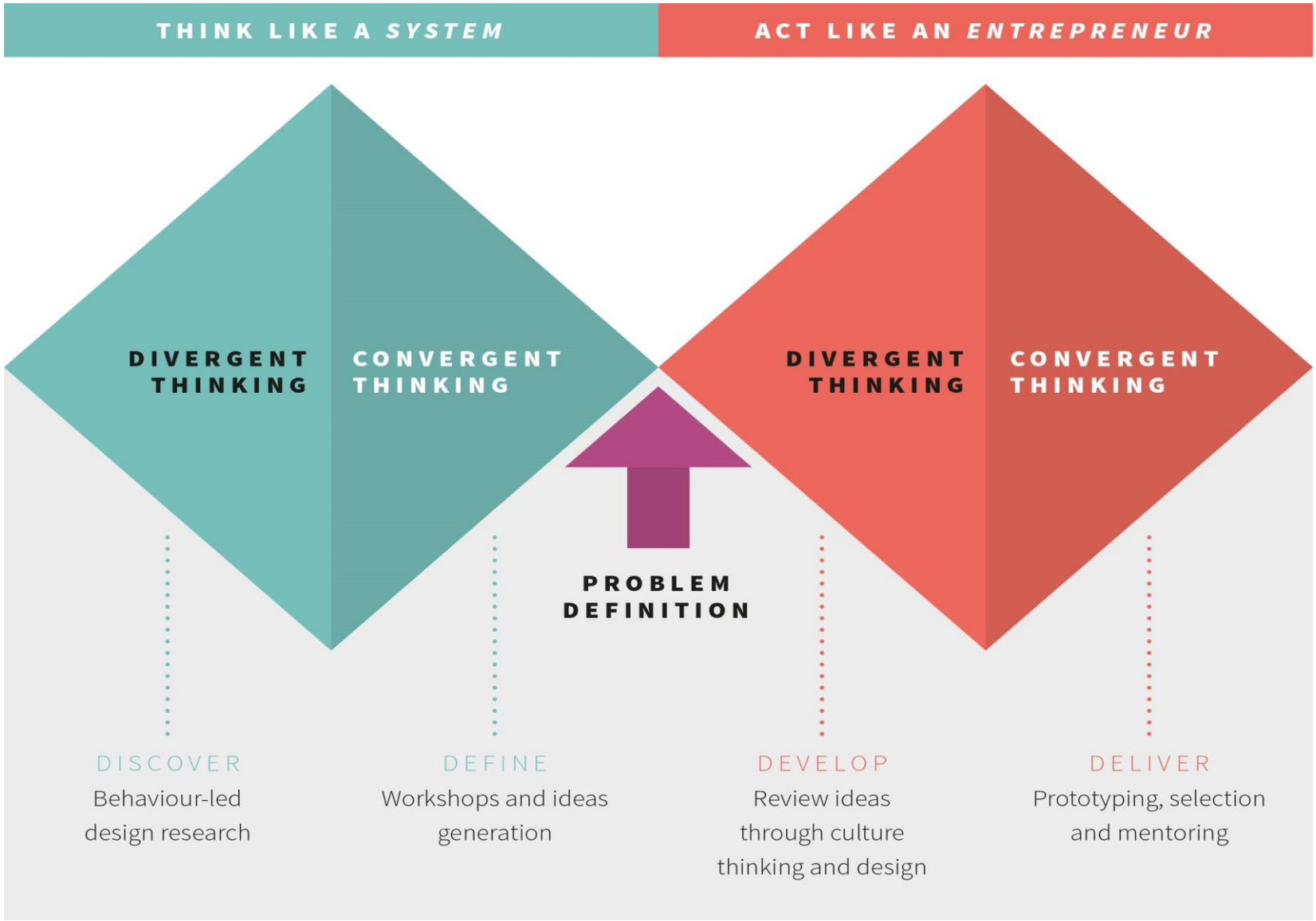
- Keep your personality
  - *“No one needs to mutate another person, because we have creativity and room for development at our disposal!*
- Love it, change it, or leave it!
  - *“Use the concepts and tips that you like and adapt them to your needs.”*
- Don’t do it alone!
  - *“Get the necessary skills, technologies, and attitudes to be successful and think in business ecosystems.”*



Optimal point in time to change the mindset..



develop new ideas….



SOLID Principles

- SOLID is a set of **five principles of object-oriented programming and design**.

- Goal: Build software that is **maintainable, extendable, and easy to understand**.

- Acronym:
- **S** – Single Responsibility Principle
- **O** – Open/Closed Principle
- **L** – Liskov Substitution Principle
- **I** – Interface Segregation Principle
- **D** – Dependency Inversion Principle



Single Responsibility Principle

- **Definition**: A class should have **only one reason to change**.
- **Non-technical Example**: A restaurant's cashier only handles billing, not cooking or cleaning. If the menu changes, the cashier isn't affected.
- **Technical Example**:
  ❌ Wrong – A class that handles both user login and sending emails:

```
class AuthService {
    login(user, password) {...}
}
class EmailService {
    sendEmail(message) {...}
}
```



Open/Closed Principle

- **Definition**: Software entities should be **open for extension, but closed for modification**.
- **Non-technical Example**: A power socket allows new appliances (extension) without needing to rebuild the entire electricity system (no modification).
- **Technical Example**:
  ❌ Wrong – Using if-else for new payment methods:

```
interface Payment {
    process()
}
class CreditCardPayment implements Payment { process() {...} }
class PayPalPayment implements Payment { process() {...} }
class PaymentProcessor {
    processPayment(payment: Payment) {
        payment.process()
    }
}
```



Liskov Substitution Principle

- **Definition**: Subclasses should be **replaceable** for their parent classes without breaking the system.
- **Non-technical Example**: If a car rental promises a "vehicle," you should be able to drive either a car, van, or truck without surprises.
- **Technical Example**:

❌ Wrong – A Penguin subclass of Bird that breaks fly() method:

```
class Bird { fly() {...} }
class Penguin extends Bird { fly() { throw Error("Can't fly") } }
```

✅ Correct – Redesign hierarchy:

```
interface Bird { eat() }
interface FlyingBird extends Bird { fly() }
class Sparrow implements FlyingBird { fly() {...} }
class Penguin implements Bird { eat() {...} }
```



Interface Segregation Principle

- **Definition**: Clients should not be forced to depend on methods they do not use.
- **Non-technical Example**: A smartphone has separate apps (camera app, messaging app). You shouldn't need to install all features in one giant app if you only want to use one.
- **Technical Example**:

❌ Wrong – A Machine interface that forces all devices to print, scan, and fax:

```
interface Machine {
   print()    scan()    fax()
}
class Printer implements Machine {
   print() {...}
   scan() {...} // not needed
   fax() {...} // not needed
}
```

✅ Correct- Split into smaller interfaces:

```
interface Printer { print() }
interface Scanner { scan() }
interface Fax { fax() }
class SimplePrinter implements Printer { print() {...} }
```



Dependency Inversion Principle

- **Definition**: Depend on **abstractions**, not on concrete implementations.
- **Non-technical Example**: A car depends on a general "fuel system," not specifically on petrol or electric—it works with any as long as the system provides energy.
- **Technical Example**:
  ❌ Wrong – High-level class depending directly on low-level class:

Correct – Use abstraction:

```
interface Database {
   save(data)
}

class MySQLDatabase implements Database { save(data) {...} }
class MongoDatabase implements Database { save(data) {...} }

class UserRepository {
   db: Database
   constructor(db: Database) { this.db = db }
   saveUser(user) { db.save(user) }
}
```



End of the Session 3



Go back to Session Overview